

Full Application Technical Manual

System Architecture

Overview

NursesAI is a full-stack educational mobile application aimed at nursing students preparing for rigorous exams like NORCET. The architecture is split between a React Native mobile client and a Node.js Express backend API, leveraging Firebase for authentication and OpenAI for intelligent tutoring.

Frontend (Mobile Application)

- **Framework:** React Native powered by Expo (SDK 54).
- **Navigation:** Expo Router providing file-based routing (`app/`).
- **Styling:** NativeWind (Tailwind CSS for React Native) paired with Global CSS.
- **Local Storage:** `@react-native-async-storage/async-storage` for retaining app state (like daily streaks and topics).

Backend (AI API Server)

- **Framework:** Node.js with Express (`backend/server.js`).
- **AI Integration:** OpenAI API (`gpt-4.1-mini`).
- **Architecture:** A stateless REST API endpoint (`POST /ask`) tailored to process nursing queries and format them into rigorous clinical structures.

Database & Authentication

- **Provider:** Firebase.
- **Components used:** `firebase/auth` and `firebase/firestore` .
- **State Management:** Live stream authentication tied to the root router layout.

Static Data Infrastructure

- Heavy JSON database approach (`data/pyqData.json`) handling 600+ static Previous Year Questions (PYQs) to minimize JS bundle compilation times and optimize offline-ready rendering.

Implementation Plan & History

Context

NursesAI has undergone significant modernization to prepare for production (APK) deployment.

Phase 1: Core Navigation & Routing Migration

- **Objective:** Move from legacy React Navigation to Expo Router.
- **Implementation:** Created the `app/` folder structure, implemented `_layout.tsx` wrapper for Firebase Auth injection, and organized screens like `dashboard`, `quiz`, and `tabs`.

Phase 2: Design Language & Gamification

- **Objective:** Establish a premium "glassmorphic" interface.
- **Implementation:** Integrated NativeWind / Tailwind CSS utilities. Added `react-native-confetti-cannon` for quiz success logic and `expo-haptics` for tactile UX reinforcement.

Phase 3: AI Processing & Context Injection

- **Objective:** Fix token truncation and enhance the AI's clinical accuracy.
- **Implementation:** Built the `backend/server.js` Express application. Configured the OpenAI API token limit to 1500 `max_tokens` and engineered strict response schemas (Definition, Causes, UI/UX, Signs/Symptoms).

Phase 4: Big Data PYQ Structuring

- **Objective:** Implement 600+ NORCET Previous Year Questions.
- **Implementation:** Formatted PDF extractions into `data/pyqData.json`. Connected it to the Quiz component which identifies array properties dynamically.

Data Flow & Logic

1. Authentication Flow

1. App Mounts -> `app/_layout.tsx` triggers `onAuthStateChanged()` .
2. Firebase verifies token.
3. If valid -> redirects to `/(tabs)` .
4. If invalid/null -> redirects to `login.tsx` .

2. Main AI Tutor Interaction Flow

1. User types message in the `AskAI` input field.
2. Component sends a `POST` request using `axios` to `http://localhost:3000/ask` .
3. Request Payload: `{ "messages": [{role: "user", content: "..."}], "mode": "standard" }` .
4. **Backend Processing:** Express server extracts messages, strips internal UI tags like "Thinking...", and prepends a rigid system prompt demanding clinical formatting.
5. Server calls `openai.chat.completions.create` .
6. Raw response returned to frontend, updating React state and rendering via `NativeWind` formatted text blocks.

3. Quiz Game Flow

1. User selects a module in the app (e.g. Medical-Surgical).
2. App reads `pyqData.json` and loads specific arrays matching the module.
3. Displays questions. When chosen, records local correct/incorrect state.
4. On finish, triggers `react-native-confetti-cannon` over the screen and updates Async Storage.

API Specifications

Local AI Server `POST /ask`

URL: `http://localhost:3000/ask` (Usually mapped to network IP for physical devices) **Description:** Takes a message array and a mode flag, returning a highly-structured clinical response.

Request Body (JSON)

```
{
  "messages": [
    {"role": "user", "content": "Explain Myocardial Infarction"}
  ],
  "mode": "standard | quiz | summary"
}
```

Modes

- `standard` : Returns deep clinical breakdown (Topic, Definition, Causes, Symptoms, Nursing Management, Complications).
- `quiz` : Returns 5 Medical MCQs with answers.
- `summary` : Returns brief bullet point notes.

Response (JSON) Success

```
{
  "answer": "🔗 Topic:\nMyocardial Infarction...\n\n📌 Definition:..."
}
```

Error Response

```
{
  "error": "Messages missing"
}
```

File Summaries

app/_layout.tsx

The primary entry index for Expo Router. Handles embedding the Inter fonts and performing the Firebase authentication state check before releasing the route lock.

firebase.ts

Initializes Firebase Auth and Firestore. Specifically modified to handle cross-platform persistence gracefully on both Expo Web (`browserLocalPersistence`) and Native app (`getReactNativePersistence`).

backend/server.js

The Express middleware providing a proxy bridge to OpenAI to prevent shipping raw API keys inside the client APK. Employs heavy custom system prompts tuned for clinical accuracy.

data/pyqData.json

A lightweight local static database holding 600+ structural NORCET clinical questions. Avoids reliance on cloud database reads and speeds up the application significantly.

package.json

Manages the massive dependency ecosystem including core (Expo, React Native), UX (Confetti Cannon, Haptics), Navigation (React Navigation, Expo Router), and Styling (NativeWind, Tailwind).